



CS & IT ENGINEERING



Algorithms

Divide & Conquer

Lecture No.- 03



By- Aditya sir

Recap of Previous Lecture



Topic

Topic

Min Max Algo
Binary Search

Topics to be Covered



Topic

Topic

Topic

Binary Search
Merge Sort



About Aditya Jain sir

1. Appeared for GATE during BTech and secured AIR 60 in GATE in very first attempt - City topper
2. Represented college as the first Google DSC Ambassador.
3. The only student from the batch to secure an internship at Amazon. (9+ CGPA)
4. Had offer from IIT Bombay and IISc Bangalore to join the Masters program
5. Joined IIT Bombay for my 2 year Masters program, specialization in Data Science
6. Published multiple research papers in well known conferences along with the team
7. Received the prestigious excellence in Research award from IIT Bombay for my Masters thesis
8. Completed my Masters with an overall GPA of 9.36/10
9. Joined Dream11 as a Data Scientist
10. Have mentored 12,000+ students & working professions in field of Data Science and Analytics
11. Have been mentoring & teaching GATE aspirants to secure a great rank in limited time
12. Have got around 27.5K followers on Linkedin where I share my insights and guide students and professionals.



Telegram Link for Aditya Jain sir: https://t.me/AdityaSir_PW



Topic : Linear Search



Linear Search:-

Algo A1 Linear (A, n, x)

For (i = 1; i ≤ n; i++)

if (A[i] == x)

{

return i

}

Print (" Not found")



key = 5



Topic : Linear Search



Linear Search TC Analysis:-

(1) Best case: $A = \begin{bmatrix} 2 & 20 & 1 & 10 \end{bmatrix}$ $x = 2$

(2) Worst case: $A = \begin{bmatrix} 2 & 5 & 10 & 7 \end{bmatrix}$

Always work

$x = 7$
↓
elem at
last positive

or $x = 8$
↓
Not
present



Topic : Linear Search



Does linear search take any advantage when the input array is sorted?

→ No

Here are need Binary Search

$$O(\log n) \leq \text{Binary Tree Height} \leq O(n)$$



Topic : Binary Search



Binary search → Divide & Conquer

Pre-requisite: Input array should be in sorted order (Mandatory Requirement)

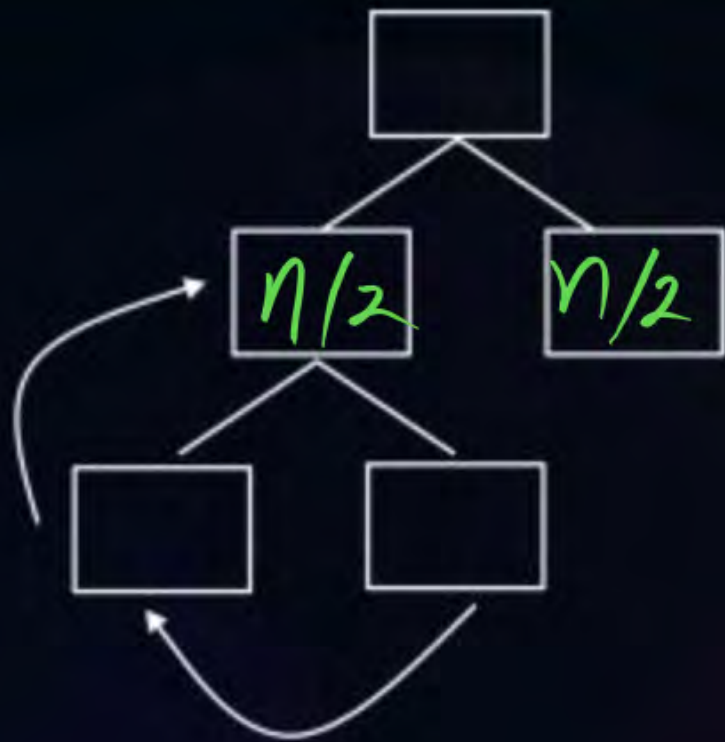
→ No such requirement in linear search.



Topic : Min-Max



$$\text{Min-max } T(n) = 2T(n/2) + 2$$





Recursion:





Topic : Min-Max



Recursion:

$\left\{ \begin{array}{ll} C1: \underline{Key = A[m]} & \rightarrow \text{Small prob sorted} \rightarrow \underline{\text{return ans}} \\ C2: \underline{Key > A[m]} & \rightarrow \text{Explore BS}(\underline{m + 1}, \underline{n}, \text{key}) \\ C3: \underline{key < A[m]} & \rightarrow \text{Explore BS}(1, \underline{m - 1}, \text{key}) \end{array} \right.$

left

Topic : Min-Max

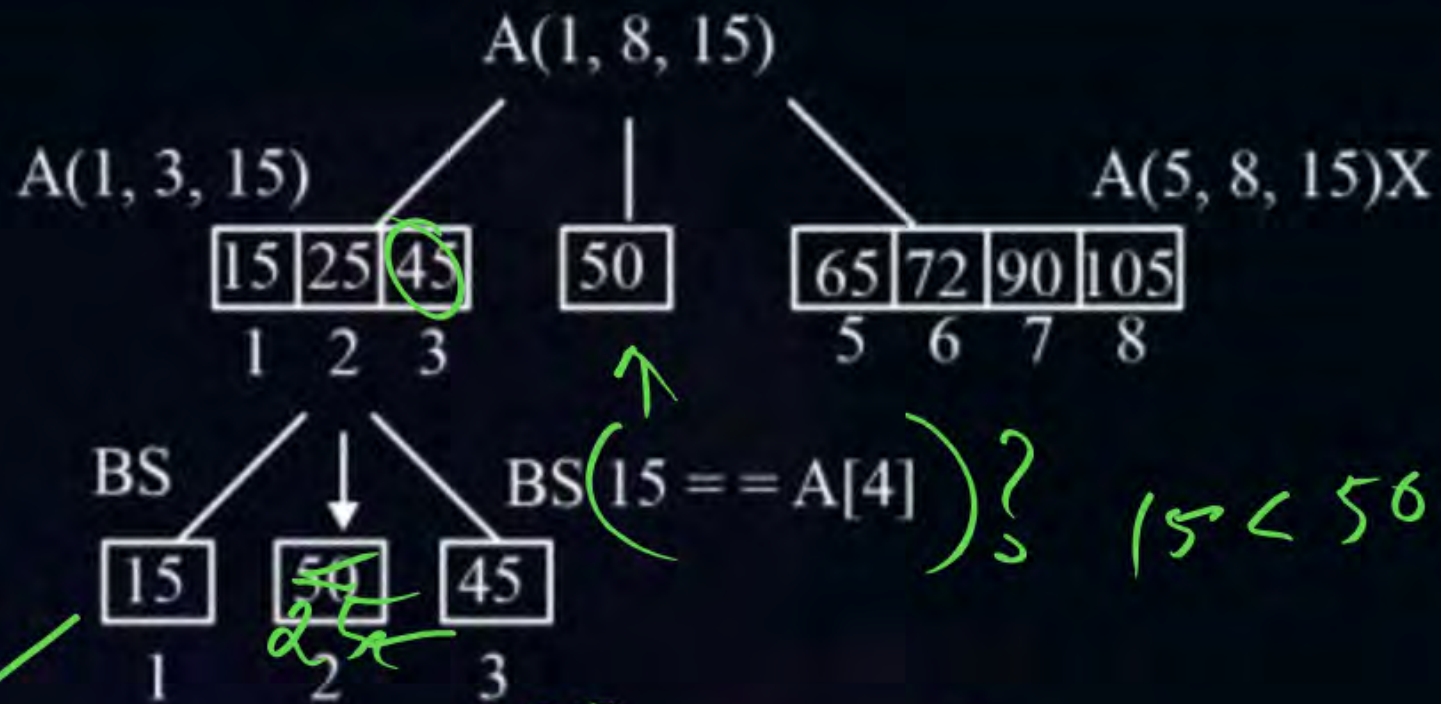


Eg.

15	25	45	50	65	72	90	105
1	2	3	4	5	6	7	8

Key = 15

Idea:



1, 8

$$\left\lfloor \frac{1+8}{2} \right\rfloor = \underline{\underline{4}}$$

{We will either ~~recursively~~ explore left subproblem or right subproblem
BUT here both}

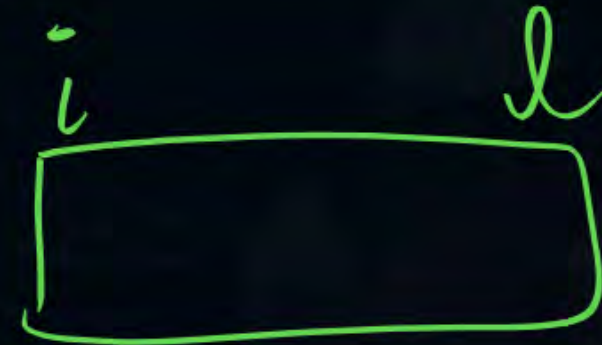
NOT



Topic : Divide and Conquer

Recur

1. Algorithm BinSrch(a, i, l, x)
2. // Given an array a[i: l] of elements in non decreasing
3. // order, $1 \leq i \leq l$, determine whether x is present, and
4. // if so, return j such that $x = a[j]$; else return 0.
5. {
6. if ($l \leq i$) then // If Small(P)
7. {
8. if ($x \leq a[i]$) then return i;
9. else return 0;
10. }





Topic : Divide and Conquer



```
11.  else
12.  { // Reduce P into a smaller subproblem.
13.  mid := [(i + l)/2];
14.  if (x = a[mid]) then return mid;
15.  else if (x < a[mid]) then
16.      return BinSrch(a, i, mid - 1, x);
17.  else return BinSrch(a, mid + 1, l, x);
18.  }
19.  }
```





Topic : Divide and Conquer



Logic:

$$T(n) = T(n/2) + C, n > 1$$

$$= b, n = 1$$

$$T(n/2) = T(n/2^2) + C$$

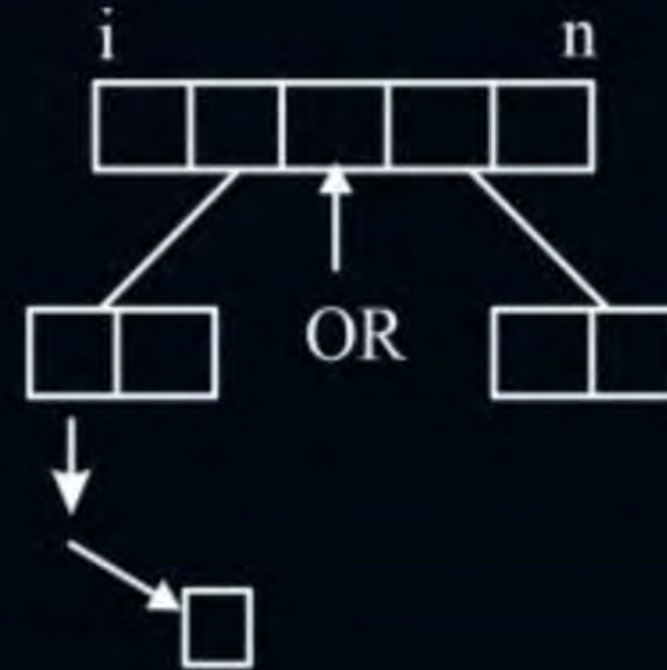
$$T(n) = T(n/2^2) + 2C$$

$$= T(n/2^3) + 3C$$

General:

$$T(n) = (n/2^k) + k * c$$

$$\text{for B.C, } \frac{n}{2^k} = 1 \Rightarrow 2^k = n \Rightarrow k = \log_2 n$$



$$T(n) = T(1) + k * c$$
$$= b + c * \log_2 n$$

$$T(n) = O(\log_2 n)$$

Best Case TC : $\Omega(1)$

Key = a[mid]





Topic : Divide and Conquer

Hence, In Binary search there is only divide but not combine no need





Topic : Divide and Conquer

Non-recursive/Iterative

Algo Binary search (A, n, x)

// A[1..n] → non-decreasing order

// return index of x, else 0.

Low = 1, high = n

While (low ≤ high)

mid = $\left\lfloor \frac{\text{low} + \text{high}}{2} \right\rfloor$

if (A[mid] > x)

high = mid - 1

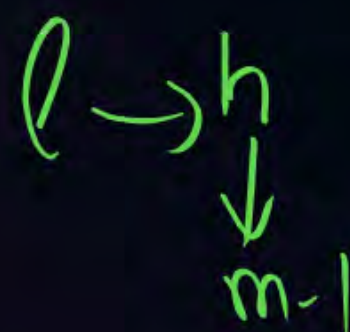
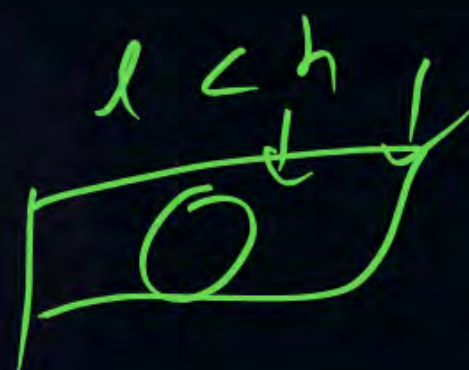
else if (A[mid] < x)

low = mid + 1

else

Return mid

or return 0



$l = m + 1$



$l > h$

Search space [low → High]





Topic : Divide and Conquer



Space Complexity analysis

(1) Non-Recursive

$$\Rightarrow \underline{SC = O(1)}$$

(2) Recursive

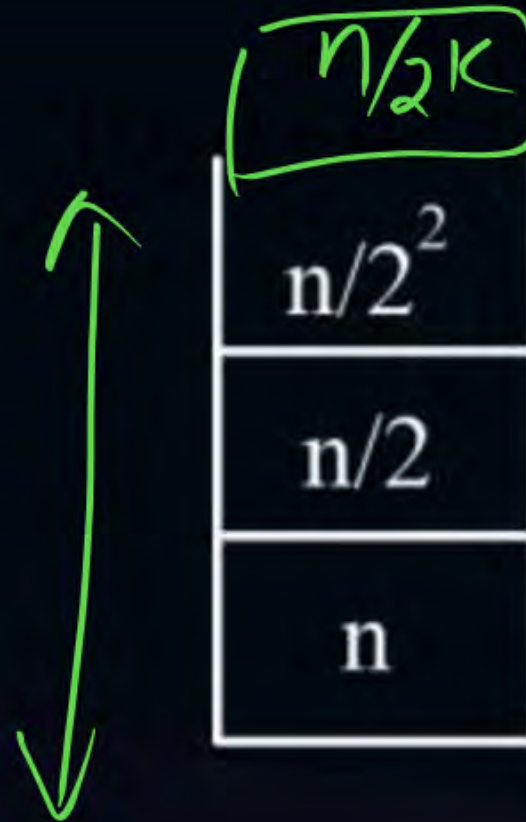
$$SC = O(\log_2 n)$$



Recursion stacks

$$k = \log_2 n$$

$$n/2^k = 1$$



Sorted array

	L.S	B.S
B.C	$\Omega(1)$	$\Omega(1)$
W.C	$O(n)$	$O(\log_2 n)$

→ BS ✓

③ Merge Sort ($V = \log n$)



Topic : Divide and Conquer

Merge Sort

(3) Merge sort

Divide & Conquer based sorting

Technique

Conquer → Merging Algo
(Principle of Merging)

→ (V.V. Imp)

Optimal Merge Pattern

Greedy Algo

1) OMP

2) Huffman Encoding



Topic : Divide and Conquer

Principle of Merging (Conquer)

Problem Statement:

$L1(m) \& L2(n)$

Given two sorted lists/Arrays ~~$L1 \& L2$~~ & such that $m \leq n$, Merge them into a single sorted list/Array $\rightarrow$ Size $\Rightarrow$ $(m + n)$

(2-way merging)

by default

min Time/Comparisons to do this?

L1 = 2, 7, 5, 3, 9



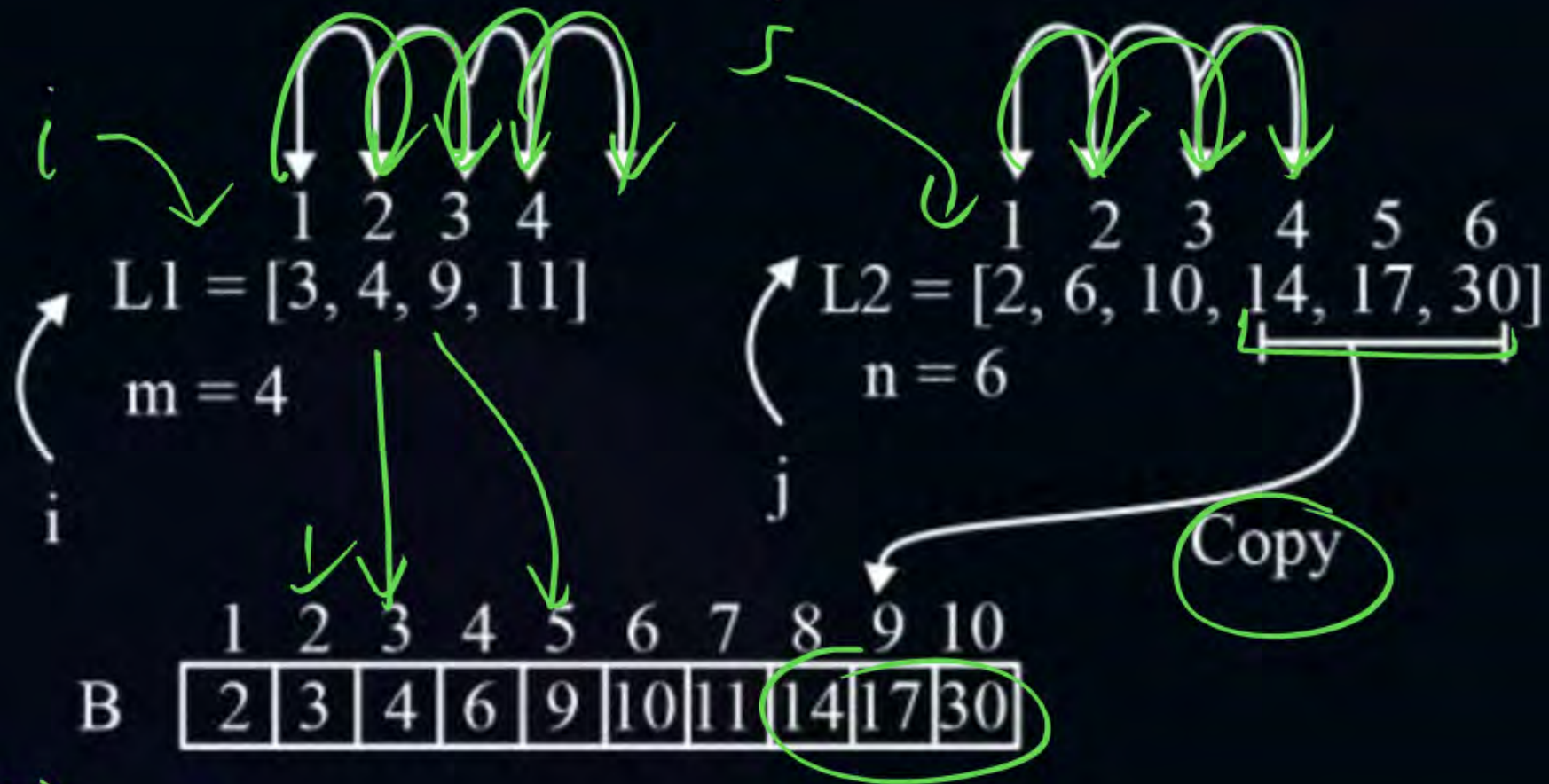
Invalid T/p

L2 = 1, 5, 7, 10, 15, 20, 22



Topic : Divide and Conquer

Eg.



$a(i)$ $a(j)$

↑
Sorted



Topic : Min-Max Problem

#Q. What are the min & max number of comparisons ^{read} ~~read~~ in 2-way merging?

Sorted L1 $\rightarrow$ M L2 $\rightarrow$ ~~n~~ ⁿ & $m \leq n$

A

$m, m + n$

B

$n, m+n-1$

C

$m, m + n - 1$

D

$n, m + n$



Topic : Divide and Conquer

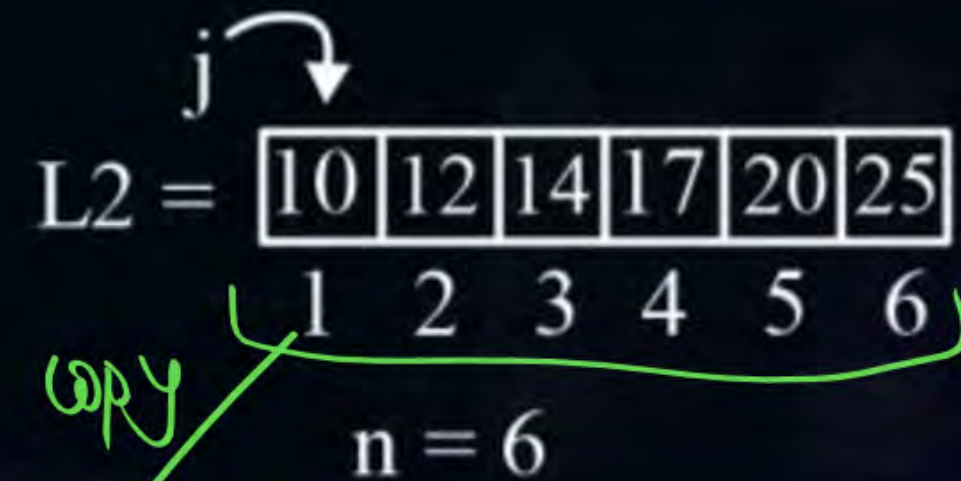
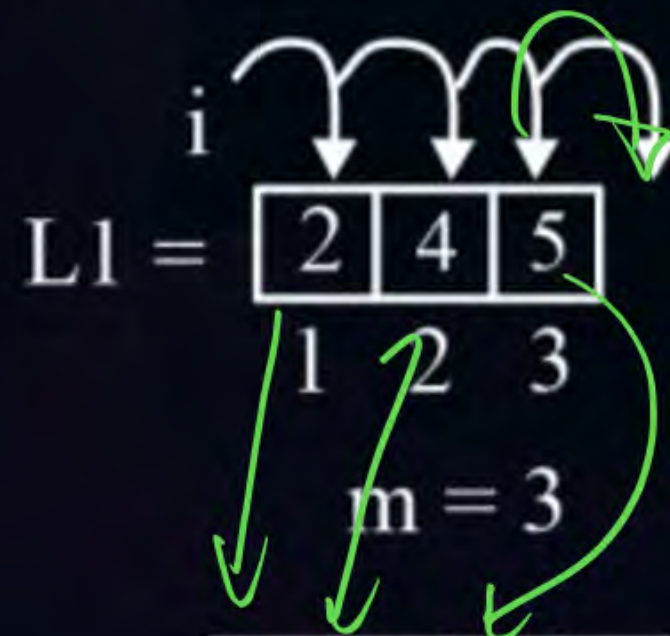
(1) Min comparisons::

Input: When all elems of smaller list (L1)_m are lesser than all the elems of larger list (L2)_n

OR

When last elem of L1 < first elem of L2

Comparisons
= m
=





Topic : Divide and Conquer

(2) Max comparisons::

V.V. Imp

Input: Smaller List (L1): $(1 \rightarrow m - 1) < \text{first elem of larger list (L2)}$

and m^{th} elem of L1 (last elem) $> \text{all elems of L2 (last elem of L2)}$

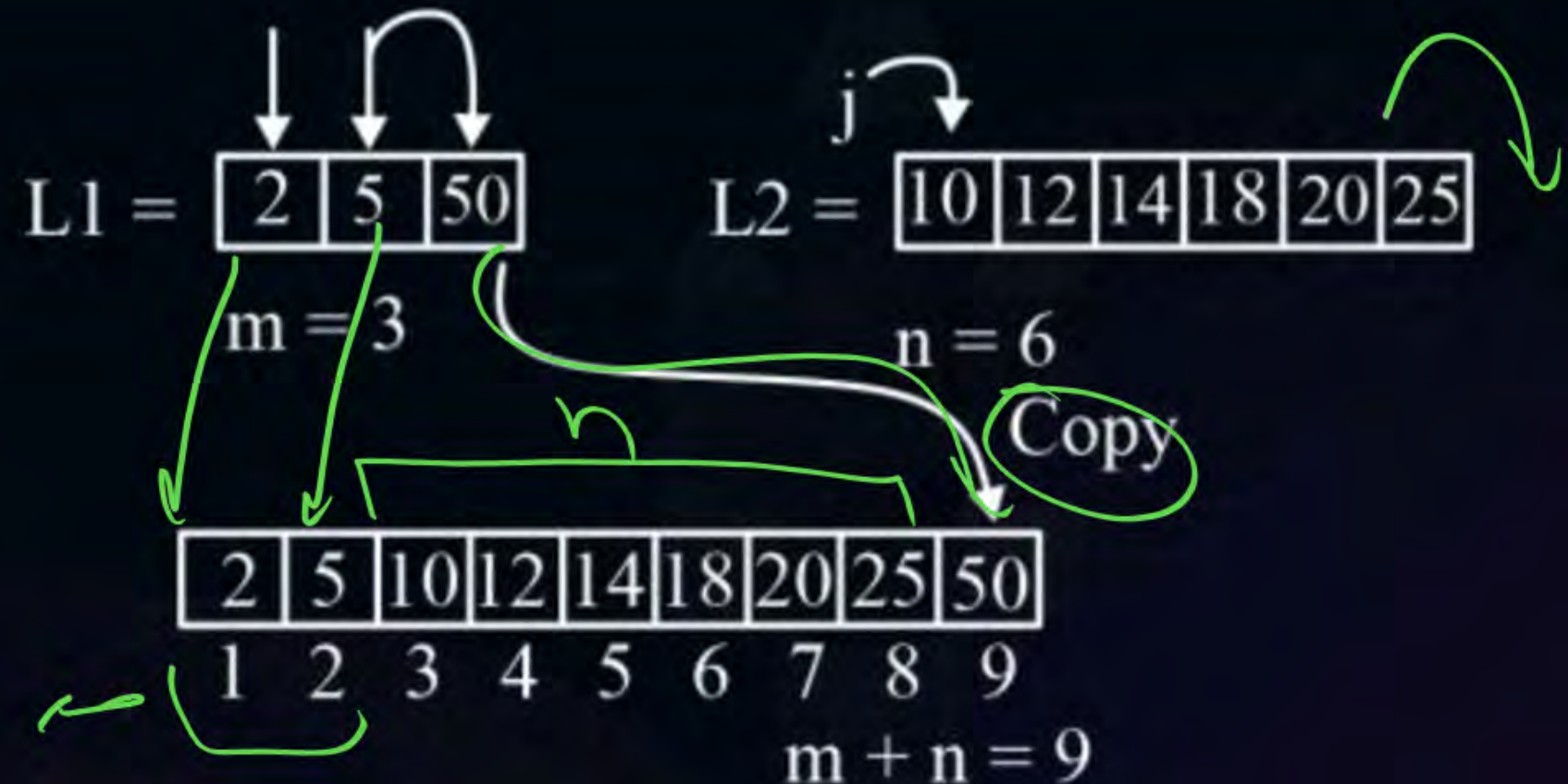
Total elem comparisons

$$= (m - 1) + n$$

$$= (m + n - 1)$$

max comparisons

V.V. Imp





Topic : Divide and Conquer

Summary:-

The number of elem comparisons reqd to merge two sorted list into a single sorted list ranges between

$$m \leq \text{comp} \leq (m + n - 1)$$

$$m \leq n$$

Sorted

L1 $\rightarrow$ m

L2 $\rightarrow$ n

~~L3 $\rightarrow$ n~~

$$m \leq n$$

$$\min(m, n) \leq \text{no. of comp} \leq (m + n - 1)$$



2 mins Summary



Topic

Topic

Topic

Topic

Topic

Binary Search
Merge Algo



THANK - YOU